

## Pamiątki

Amaru kupuje pamiątki w sklepie za granicą. Istnieje  $N$  **typów** pamiątek. W sklepie dostępna jest nieskończona liczba pamiątek każdego typu.

Każdy typ pamiątki ma ustaloną cenę. Pamiątka typu  $i$  ( $0 \leq i < N$ ) kosztuje  $P[i]$  monet, gdzie  $P[i]$  jest liczbą całkowitą dodatnią.

Amaru wie, że typy pamiątek są posortowane malejąco według ceny, i że ceny pamiątek są parami różne. Bardziej formalnie:  $P[0] > P[1] > \dots > P[N-1] > 0$ . Co więcej, udało mu się poznać wartość  $P[0]$ . Niestety, Amaru nie posiada żadnych innych informacji na temat cen pamiątek.

Aby kupić pamiątki, Amaru przeprowadzi szereg transakcji ze sprzedawcą.

Każda transakcja składa się z następujących kroków:

1. Amaru wręcza sprzedawcy pewną (dodatnią) liczbę monet.
2. Sprzedawca układa te monety w stos na stole na zapleczu, gdzie Amaru nie może ich zobaczyć.
3. Sprzedawca rozpatruje każdy typ pamiątki  $0, 1, \dots, N-1$  w tej kolejności, jeden po drugim. Każdy typ jest rozpatrywany **dokładnie raz** dla każdej transakcji.
  - Rozważając typ pamiątki  $i$ , jeśli aktualna liczba monet na stosie wynosi co najmniej  $P[i]$ , to
    - sprzedawca usuwa  $P[i]$  monet ze stosu, a następnie
    - sprzedawca kładzie na stole jedną pamiątkę typu  $i$ .
4. Sprzedawca przekazuje Amaru wszystkie monety pozostałe na stosie i wszystkie pamiątki znajdujące się na stole.

Zwróć uwagę, że przed rozpoczęciem transakcji na stole nie znajdowały się żadne monety ani pamiątki.

Twoim zadaniem jest poinstruowanie Amaru, aby wykonał określoną liczbę transakcji, tak aby:

- w każdej transakcji kupował **przynajmniej jedną** pamiątkę, oraz
- łącznie kupić **dokładnie**  $i$  pamiątek typu  $i$ , dla każdego  $i$  takiego, że  $0 \leq i < N$ . Zwróć uwagę, że w szczególności oznacza to, iż Amaru nie powinien kupić żadnych pamiątek typu 0.

Amaru nie musi minimalizować liczby transakcji.

## Szczegóły implementacji

Należy zaimplementować następującą procedurę.

```
void buy_souvenirs(int N, long long P0)
```

- $N$  : liczba typów pamiątek.
- $P0$  : wartość  $P[0]$ .
- Ta procedura jest wywoływana dokładnie raz dla każdego przypadku testowego.

Powyższa procedura może wywoływać następującą procedurę, żeby polecić Amaru wykonanie transakcji:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- $M$ : liczba monet wręczonych sprzedawcy przez Amaru.
- Procedura zwraca parę. Pierwszym elementem pary jest tablica  $L$ , zawierające typy zakupionych pamiątek (w kolejności rosnącej). Drugim elementem jest liczba całkowita  $R$ , liczba monet zwróconych Amaru po transakcji.
- Wymagane jest, aby  $P[0] > M \geq P[N - 1]$ . Warunek  $P[0] > M$  zapewnia, że Amaru nie kupi żadnej pamiątki typu 0, a  $M \geq P[N - 1]$  zapewnia, że Amaru kupi co najmniej jedną pamiątkę. Jeśli te warunki nie zostaną spełnione, Twoje rozwiązanie otrzyma werdykt `Output isn't correct: Invalid argument`. Należy zauważyć, że w przeciwieństwie do  $P[0]$ , wartość  $P[N - 1]$  nie jest podana na wejściu.
- Procedurę można wywołać maksymalnie 5000 w dla każdym przypadku testowym.

Programu oceniający **nie jest adaptacyjny**. Oznacza to, że sekwencja cen  $P$  jest ustalona przed wywołaniem procedury `buy_souvenirs`.

## Ograniczenia

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$  dla każdego  $i$  takiego, że  $0 \leq i < N$ .
- $P[i] > P[i + 1]$  dla każdego  $i$  takiego, że  $0 \leq i < N - 1$ .

## Podzadania

| Podzadanie | Punktacja | Dodatkowe ograniczenia                                                                                                                                            |
|------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1          | 4         | $N = 2$                                                                                                                                                           |
| 2          | 3         | $P[i] = N - i$ dla każdego $i$ takiego, że $0 \leq i < N$ .                                                                                                       |
| 3          | 14        | $P[i] \leq P[i + 1] + 2$ dla każdego $i$ takiego, że $0 \leq i < N - 1$ .                                                                                         |
| 4          | 18        | $N = 3$                                                                                                                                                           |
| 5          | 28        | $P[i + 1] + P[i + 2] \leq P[i]$ dla każdego $i$ takiego, że $0 \leq i < N - 2$ .<br>$P[i] \leq 2 \cdot P[i + 1]$ dla każdego $i$ takiego, że $0 \leq i < N - 1$ . |
| 6          | 33        | Brak dodatkowych ograniczeń.                                                                                                                                      |

## Przykład

Rozważmy następujące wywołanie.

```
buy_souvenirs(3, 4)
```

Mamy  $N = 3$  typy pamiątek i  $P[0] = 4$ . Zauważ, że istnieją tylko trzy możliwe sekwencje cen  $P$ :  $[4, 3, 2]$ ,  $[4, 3, 1]$ , i  $[4, 2, 1]$ .

Założmy, że `buy_souvenirs` wywołuje `transaction(2)`. Założmy, że wywołanie zwraca  $([2], 1)$ , co oznacza, że Amaru kupił jedną pamiątkę typu 2, a sprzedawca oddał mu 1 monetę. Zauważmy, że pozwala nam to wywnioskować, że  $P = [4, 3, 1]$ , ponieważ:

- Dla  $P = [4, 3, 2]$ , `transaction(2)` zwróciłoby  $([2], 0)$ .
- Dla  $P = [4, 2, 1]$ , `transaction(2)` zwróciłoby  $([1], 0)$ .

Następnie `buy_souvenirs` może wywołać `transaction(3)`, co zwraca  $([1], 0)$ , co oznacza, że Amaru kupił jedną pamiątkę typu 1, a sprzedawca oddał mu 0 monet. Do tej pory łącznie kupił jedną pamiątkę typu 1 i jedną pamiątkę typu 2.

Na koniec `buy_souvenirs` może wywołać `transaction(1)`, co zwraca  $([2], 0)$ , co oznacza, że Amaru kupił jedną pamiątkę typu 2. Zauważ, że w tej sytuacji moglibyśmy również wywołać `transaction(2)`. W tym momencie Amaru łącznie kupił jedną pamiątkę typu 1 i dwie pamiątki typu 2, zgodnie z wymaganiami.

## Przykładowy program oceniający

Format wejściowy:

$N$

$P[0] \ P[1] \ \dots \ P[N-1]$

Format wyjściowy:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Gdzie  $Q[i]$  jest liczbą łącznie kupionych pamiątek typu  $i$ , dla każdego  $i$  takiego, że  $0 \leq i < N$ .